GREEDY ALGORITHMS –

Greedy algorithms form one of the foundational paradigms of algorithmic design, relying on a deceptively simple idea: at every step of the computation, choose the option that looks best immediately, without reconsidering earlier decisions. Despite its apparent simplicity, the greedy method succeeds beautifully on a wide class of optimization problems that satisfy two critical properties: the greedy-choice property and optimal substructure. The greedy-choice property asserts that a locally optimal decision at a given stage is always part of some global optimum. Complementing this, optimal substructure ensures that optimal solutions to subproblems compose an optimal solution to the whole problem. When both conditions hold, greedy algorithms often yield solutions that are not only correct but elegant and efficient.

The slides for this module introduce greedy algorithms as a natural extension of human instinct: when faced with many options, choose the one that appears most advantageous right now. But unlike intuition, the algorithmic greedy method requires rigorous justification. Many problems reject the greedy approach because the best immediate-looking choice leads to dead ends or suboptimal outcomes. Yet in the problems where it fits, the greedy paradigm offers some of the simplest and fastest possible solutions.

The provided generic greedy template captures the essence of the strategy. It begins with an empty solution and repeatedly augments it by selecting an element that appears best according to some problem-specific criterion. A feasibility test ensures that adding the chosen element does not violate the constraints of the problem. The process continues until all input elements have been considered. Although concise, this skeletal structure underpins myriad famous algorithms, including Dijkstra's shortest path algorithm, Prim's and Kruskal's minimum spanning tree algorithms, Huffman coding, job scheduling, and many others.

The overarching pseudocode, as shown in the slides, reads:

```
Greedy(a, n)
// a[1..n] contains the n inputs
{
    solution = ∅;
    for i = 1 to n do
    {
        x = Select(a);
        if Feasible(solution, x) then
            solution = Union(solution, x);
    }
    return solution;
}
```

The power of this template lies in the choice of the **Select** function: it encapsulates the strategy. A poorly chosen Select function results in a useless or incorrect algorithm, while the right choice makes the greedy solution provably optimal.

SPANNING TREES AND THEIR ROLE IN GREEDY ALGORITHMS

A spanning tree is a foundational concept in graph theory and algorithms, and understanding it is crucial for the study of greedy methods. Given a connected, undirected graph G=(V,E)G = (V, E)G=(V,E), a spanning tree is a subgraph that includes all vertices of the graph but only enough edges to ensure connectivity, without forming any cycles. A graph with |V|=n|V| = n|V|=n vertices must have exactly n−1n - 1n−1 edges in any spanning tree, a fact that follows directly from the tree definition.

The slides illustrate the concept using a simple four-vertex graph. The original graph has vertices a,b,c,da, b, c, da,b,c,d with edges connecting various pairs. Two distinct spanning trees are shown to highlight that spanning trees are typically not unique. In the first spanning tree, the tree forms a simple chain: aaa is connected to bbb, which connects to ccc, which connects to ddd. In the second spanning tree, vertex aaa branches to both bbb and ddd, and ddd connects to ccc. Both subgraphs connect all vertices and use exactly three edges, but their structures differ—demonstrating that multiple spanning trees may exist for a single graph.

WEIGHTED GRAPHS AND THE MINIMUM SPANNING TREE (MST)

The concept of weights introduces optimization into spanning trees. A weighted graph assigns a real number (often representing cost, distance, or time) to each edge. The weight of a spanning tree is then defined as the sum of the weights of its edges. The **Minimum Spanning Tree (MST)** is the spanning tree whose total weight is minimum among all spanning trees of the graph.

The slides provide a detailed weighted graph to illustrate this. The graph consists of six vertices:

Vertices: a,b,c,d,e,fa, b, c, d, e, fa,b,c,d,e,f

Edges with weights (all extracted exactly from the figure):

- (a,b)=32(a, b) = 32(a,b)=32
- (a,d)=10(a, d) = 10(a,d)=10
- (b,d)=25(b, d) = 25(b,d)=25
- (b,c)=5(b, c) = 5(b,c)=5
- (d,c)=8(d, c) = 8(d,c)=8
- (d,f)=14(d, f) = 14(d,f)=14
- (c,f)=2(c, f) = 2(c,f)=2
- (f,e)=1(f, e) = 1(f,e)=1
- (d,e)=65(d, e) = 65(d,e)=65

- (b,e)=45(b, e) = 45(b,e)=45

This graph is rich in structure: it allows various spanning trees, each with different total weights. The slides also provide two example spanning trees, each containing a subset of the original edges, and use them to illustrate why MSTs may be non-unique. In one spanning tree, the chosen edges yield a total weight of 71, while another valid spanning tree has total weight 72. Both are legitimate spanning trees, but only the smallest weight—71—qualifies as the MST.

It is also possible that multiple distinct spanning trees have identical minimum weight, especially when the graph has many edges with equal or symmetric weights. This non-uniqueness underscores the need for algorithmic precision in selecting the correct edges.

THE MST PROBLEM AND GENERIC APPROACH

The MST problem asks: given a connected weighted undirected graph G=(V,E)G = (V, E)G=(V,E), find a subset of edges forming a tree of minimum total weight. A naïve approach would enumerate all possible spanning trees—but their number is exponential in the number of vertices. Instead, greedy methods deliver remarkably fast and elegant solutions.

A classic strategy begins with an empty graph. At each step, the algorithm seeks a safe edge—an edge that can be added without creating a cycle and that preserves the possibility of forming an MST. Cycle avoidance is crucial: any cycle necessarily has an edge whose weight is greater than or equal to the others, and removing the heaviest edge keeps connectivity while reducing weight.

Both **Prim's** and **Kruskal's** algorithms exploit this idea in different ways.

PRIM'S ALGORITHM – A GREEDY APPROACH TO MST

Prim's algorithm grows a minimum spanning tree one vertex at a time, always choosing the lightest edge that connects a new vertex to the evolving tree. It begins with an arbitrary vertex, which serves as the seed of the MST.

To illustrate Prim's algorithm step-by-step, the slides provide a large weighted graph with vertices arranged approximately in a circular shape. The algorithm proceeds by:

1. Initializing a set SSS containing a single start vertex rrr.
2. Selecting the minimum-weight edge that connects a vertex in SSS to a vertex not in SSS.

3. Adding the selected edge and its endpoint to SSS.
4. Repeating this process until all vertices join the tree.

The slides walk through this procedure using the weighted graph shown earlier. Suppose the algorithm begins at vertex aaa. The algorithm first chooses the lowest-cost edge incident to aaa, which is (a,d)=10(a, d) = 10(a,d)=10. Next, it looks at all edges from S={a,d}S = \{a, d\}S={a,d} to the remaining vertices: the lightest among them is now (d,c)=8(d, c) = 8(d,c)=8. The process continues, selecting edges such as (c,f)=2(c, f) = 2(c,f)=2, (f,e)=1(f, e) = 1(f,e)=1, and (b,c)=5(b, c) = 5(b,c)=5. When the algorithm terminates, the edges chosen collectively form the MST.

Throughout this sequence, the slides show diagrams depicting the frontier between selected and unselected vertices, offering a visual narrative of the MST growth.

PRIORITY QUEUE IMPLEMENTATION

To efficiently select minimum-weight edges, Prim’s algorithm uses a priority queue storing vertices along with a "key" value—the lightest weight edge connecting each vertex to the existing MST. The algorithm repeatedly extracts the vertex with minimum key value and relaxes its neighbors, updating their key values accordingly.

This implementation reduces the running time to O((V+E)log V)O((V + E) \log V)O((V+E)logV) with a binary heap and further to O(Vlog V+E)O(V \log V + E)O(VlogV+E) when a Fibonacci heap is used.

KRUSKAL’S ALGORITHM

While Prim’s algorithm grows a single tree, Kruskal’s algorithm begins with every vertex isolated and repeatedly adds the globally lightest edge that does not create a cycle.

To determine whether an edge forms a cycle, Kruskal uses a **Disjoint Set Union-Find** structure. Each vertex starts in its own set. When an edge connects two different sets, they are merged. If an edge connects vertices already in the same set, it is discarded.

The slides depict this process with a weighted graph and illustrate how the union-find structure evolves. The resulting MST consists of the edges selected in increasing weight order, provided they do not introduce cycles.

The time complexity is dominated by sorting the edges, giving O(Elog E)O(E \log E)O(ElogE).

SHORTEST PATHS – SINGLE SOURCE

Greedy ideas play a central role in shortest path algorithms as well. The slides introduce the single-source shortest path problem: given a source vertex sss, compute the minimum distance from sss to every other vertex.

Shortest paths obey the triangle inequality:

δ(u,v)≤δ(u,x)+w(x,v)\delta(u, v) \leq \delta(u, x) + w(x, v)δ(u,v)≤δ(u,x)+w(x,v)

This inequality, combined with relaxation, forms the core operation in Dijkstra's algorithm.

The relaxation operation is:

```
Relax(u, v, w):
    if d[v] > d[u] + w(u,v):
        d[v] = d[u] + w(u,v)
```

The operation updates distance estimates when a better (shorter) route is found.

DIJKSTRA'S ALGORITHM

Dijkstra's algorithm computes shortest paths in graphs with non-negative edge weights. It operates similarly to Prim's algorithm, maintaining a priority queue keyed by current distance estimates. Initially, all distances are set to infinity except for the source, which is set to zero.

The algorithm repeatedly extracts the vertex with the minimum distance and relaxes all its outgoing edges. Because weights are nonnegative, once a vertex is extracted, its shortest path is finalized.

The slides provide the canonical pseudocode:

```
Dijkstra(G):
{
    for each v in V:
        d[v] = ∞;
    d[s] = 0;
    S = ∅;
    Q = V;

    while (Q ≠ ∅):
        u = ExtractMin(Q);
        S = S ∪ {u};
        for each v in Adj[u]:
            if (d[v] > d[u] + w(u,v)):
                d[v] = d[u] + w(u,v);
}
```

With a binary heap, this runs in O((V+E)logf_0V)O((V + E) \log V)O((V+E)logV).
Dijkstra's algorithm does **not** work when negative edge weights exist.